Question 1 : Write a Python program that stores two numbers and prints:

- Sum
- Difference
- Product
- Quotient
- Floor Division
- Modulus
- Exponent

```
first = 10
second = 3

print("Sum:", first + second)
print("Difference:", first - second)
print("Product:", first * second)
try:
    print("Quotient:", first / second)
    print("Floor Division:", first // second)
    print("Modulus:", first % second)
except ZeroDivisionError:
    print("Cannot divide by zero.")
print("Exponent:", first ** second)
```

```
Sum: 13
Difference: 7
Product: 30
Quotient: 3.3333333333333335
Floor Division: 3
Modulus: 1
Exponent: 1000
```

Question 2 : Create a list of five fruits.
Perform the following:

- Add Mango
- Remove Banana
- Sort the list
- Reverse the list

```
fruits = ["Apple", "Banana", "Orange", "Grapes", "Guava"]
fruits.append("Mango")
print("After adding Mango:", fruits)
fruits.remove("Banana")
print("After removing Banana:", fruits)
fruits.sort()
print("Sorted list:", fruits)
fruits.reverse()
print("Reversed list:", fruits)
```

```
After adding Mango: ['Apple', 'Banana', 'Orange', 'Grapes', 'Guava', 'Mango']
After removing Banana: ['Apple', 'Orange', 'Grapes', 'Guava', 'Mango']
Sorted list: ['Apple', 'Grapes', 'Guava', 'Mango', 'Orange']
Reversed list: ['Orange', 'Mango', 'Guava', 'Grapes', 'Apple']
```

Question 3 : Create a dictionary containing Student Name, Age and Course.

Print all keys and values separately.

```
student = {"Student Name": "Rahul", "Age": 20, "Course": "Python"}
print("Keys:", list(student.keys()))
print("Values:", list(student.values()))
```

```
Keys: ['Student Name', 'Age', 'Course']
Values: ['Rahul', 20, 'Python']
```

Question 4 : Create four reusable functions:

- add()
- subtract()
- multiply()
- divide()

Call each function.

```
def add(a, b):
    return a + b

def subtract(a, b):
    return a - b

def multiply(a, b):
    return a * b

def divide(a, b):
    if b == 0:
        raise ZeroDivisionError("Cannot divide by zero.")
    return a / b

print("Addition:", add(10, 2))
print("Subtraction:", subtract(10, 2))
print("Multiplication:", multiply(10, 2))
try:
    print("Division:", divide(10, 2))
except ZeroDivisionError as error:
    print(error)
```

```
Addition: 12
Subtraction: 8
Multiplication: 20
Division: 5.0
```

Question 5 : Given

employee = {

"Name":"Amit",

"Department":"IT",

"Salary":50000

}

Perform:

- Add Experience
- Update Salary
- Delete Department

```
employee = {
    "Name":"Amit",
    "Department":"IT",
    "Salary":50000
}
employee["Experience"] = 3
print("After adding Experience:", employee)
employee["Salary"] = 60000
print("After updating Salary:", employee)
del employee["Department"]
print("After deleting Department:", employee)
```

```
After adding Experience: {'Name': 'Amit', 'Department': 'IT', 'Salary': 50000, 'Experience': 3}
After updating Salary: {'Name': 'Amit', 'Department': 'IT', 'Salary': 60000, 'Experience': 3}
After deleting Department: {'Name': 'Amit', 'Salary': 60000, 'Experience': 3}
```

Question 6 : Import the math module and calculate

- Square root of 144
- Factorial of 6
- Value of π

```
import math
```

```
print("Square root of 144:", math.sqrt(144))
print("Factorial of 6:", math.factorial(6))
print("Value of π:", math.pi)
```

```
Square root of 144: 12.0
Factorial of 6: 720
Value of π: 3.141592653589793
```

Question 7 : Generate an 8-digit random password using the random module.

```
import random

password = random.randint(10000000, 99999999)
print("Password:", password)
```

```
Password: 25833629
```

Question 8 : Write a program that asks the user to enter two numbers.
Handle division by zero.

```
try:
    first = float(input("Enter first number: "))
    second = float(input("Enter second number: "))
    print("Quotient:", first / second)
except ZeroDivisionError:
    print("Cannot divide by zero.")
except ValueError:
    print("Please enter valid numbers.")
```

```
Enter first number: 10
Enter second number: 2
Quotient: 5.0
```

Question 9 : Given
data=[2,3,4,2,5,2,3,6,5]
Create a dictionary containing the frequency of each element.

```
data=[2,3,4,2,5,2,3,6,5]
frequency = {}
for number in data:
    frequency[number] = frequency.get(number, 0) + 1
print(frequency)
```

```
{2: 3, 3: 2, 4: 1, 5: 2, 6: 1}
```

Question 10 : Write a function that accepts marks of five subjects and returns

- Total
- Percentage
- Grade

```
def calculate_result(marks):
    # Each subject is out of 100; grades: A >= 90, B >= 80, C >= 70,
    # D >= 60, E >= 50, and F below 50.
    if len(marks) != 5 or any(mark < 0 or mark > 100 for mark in marks):
        raise ValueError("Enter five marks between 0 and 100.")
    total = sum(marks)
    percentage = total / 500 * 100
    if percentage >= 90:
        grade = "A"
    elif percentage >= 80:
        grade = "B"
    elif percentage >= 70:
        grade = "C"
    elif percentage >= 60:
        grade = "D"
    elif percentage >= 50:
```

```
            grade = "E"
        else:
            grade = "F"
        return total, percentage, grade

    try:
        total, percentage, grade = calculate_result([85, 90, 78, 88, 84])
        print("Total:", total)
        print("Percentage:", percentage)
        print("Grade:", grade)
    except ValueError as error:
        print(error)
```

```
Total: 425
Percentage: 85.0
Grade: B
```

Question 11 : Given
employees = {
"Amit":45000,
"Neha":55000,
"Riya":70000,
"Raj":65000
}
Write a program to

- Increase every salary by 10%
- Find highest-paid employee
- Find average salary

```
employees = {
    "Amit":45000,
    "Neha":55000,
    "Riya":70000,
    "Raj":65000
}
for name in employees:
    employees[name] = round(employees[name] * 1.10, 2)
print("Updated salaries:", employees)
highest_paid = max(employees, key=employees.get)
print("Highest-paid employee:", highest_paid)
print("Average salary:", sum(employees.values()) / len(employees))
```

```
Updated salaries: {'Amit': 49500.0, 'Neha': 60500.0, 'Riya': 77000.0, 'Raj': 71500.0}
Highest-paid employee: Riya
Average salary: 64625.0
```

Question 12 : Given
students = {
"Rahul":{"Math":90,"Science":88},
"Aman":{"Math":78,"Science":80},
"Neha":{"Math":95,"Science":92}
}
Display

- Total marks
- Average marks
- Top score

```
students = {
    "Rahul":{"Math":90,"Science":88},
    "Aman":{"Math":78,"Science":80},
    "Neha":{"Math":95,"Science":92}
}
totals = {}
for name, marks in students.items():
    total = sum(marks.values())
    totals[name] = total
    print(name, "- Total marks:", total, "Average marks:", total / len(marks))
```

```
top_student = max(totals, key=totals.get)
print("Top score:", totals[top_student], "-", top_student)
```

```
Rahul - Total marks: 178 Average marks: 89.0
Aman - Total marks: 158 Average marks: 79.0
Neha - Total marks: 187 Average marks: 93.5
Top score: 187 - Neha
```

Question 13 : Generate a random 6-digit OTP.

Ask the user to enter the OTP.

Display whether verification is successful.

```
import random

otp = str(random.randint(100000, 999999))
print("OTP:", otp)
entered_otp = input("Enter the OTP: ")
if entered_otp == otp:
    print("Verification successful.")
else:
    print("Verification failed.")
```

```
OTP: 374879
Enter the OTP: 374879
Verification successful.
```

Question 14 : Create reusable functions

- Deposit
- Withdraw
- Check Balance

Handle insufficient balance using exception handling.

```
def deposit(balance, amount):
    if amount <= 0:
        raise ValueError("Amount must be positive.")
    return balance + amount

def withdraw(balance, amount):
    if amount <= 0:
        raise ValueError("Amount must be positive.")
    if amount > balance:
        raise ValueError("Insufficient balance.")
    return balance - amount

def check_balance(balance):
    return balance

balance = 1000
try:
    balance = deposit(balance, 500)
    print("Balance after deposit:", check_balance(balance))
    balance = withdraw(balance, 300)
    print("Balance after withdrawal:", check_balance(balance))
    balance = withdraw(balance, 2000)
except ValueError as error:
    print(error)
print("Current balance:", check_balance(balance))
```

```
Balance after deposit: 1500
Balance after withdrawal: 1200
Insufficient balance.
Current balance: 1200
```